Unit 1

Introduction [5 Hrs]

1.1 Concept of procedure , object, event oriented language ,aspect-oriented programming subject oriented programming:

a) Procedure Oriented Programming/Structured Programming

- ✓ A procedural program is written as a list of instructions, telling the computer, step-by-step, what to do: Get two numbers, add them and display the sum.
- ✓ Procedural programming is fine for small projects. It does not model real world problems well.
- ✓ The traditional programming languages like C, FORTRAN, Pascal, and Basic are Procedural Programming Languages.

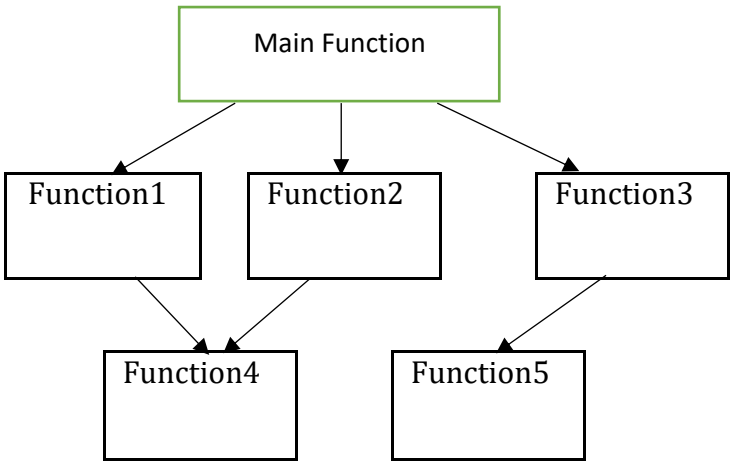


Fig: Typical structure of Procedural Oriented Program

❖ **Features of Structured Programming Language:**

- ✓ In multi-function program, data items that are important and that are used by more than one functions, are placed as global so that they may be accessed by all the functions.
- ✓ Data move openly around the system from function to function.
- ✓ Information is passed between functions using parameters or arguments.
- ✓ Emphasis is on algorithm (step by step description of problem solving) to solve a problem.
- ✓ Large program is divided into a number of functions, with each function having a clearly defined purpose and a clearly defined interface to other functions.

b) Object Oriented Programming :

Object oriented programming is a programming paradigm that was developed to overcome the drawbacks and limitations of particularly procedure oriented programming.

Examples: Simula, Modula, Smalltalk, Ada, Objective-C, CLOS (Common Lisp Object Standard), Standard C++, Java, Scripting languages: Perl, JavaScript, Python

➢ Characteristics of OOP

- Emphasis is on data rather than procedure.
- Programs are divided into objects
- Data structures are designed such that they characterize the objects.
- Functions that separate on the data of an object are tied together in the data structure.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through functions.
- New data and functions can be easily added whenever necessary.
- Bottom-up approach is followed in program design.

➢ Advantages of OOP:

- We can eliminate redundant codes by using inheritance feature of OOP.
- It is very easy for managing complex and large size problems.
- The most important is the reusability of codes by using the features inheritance.
- It takes very less time for the development and maintaining the software.
- It is efficient for testing and implementation of the system.
- It follows bottom up approach.
- It can be implemented in the field of OODBMS, OOAD and different fields of engineering.

➢ Disadvantage of OOP

- Over generalization
- Artificial class relations
- Unnecessary complications
- Size: Object oriented programs are much larger than other programs.
- Effort: Object oriented programs require a lot of work to create.
- Speed: Object oriented programs are slower than other programs, partially because of their size.

❖ Features of OOP:

- **Object:** An object is any entity, thing or organization that exits in real world. It consists of two fundamentals characteristics: its attributes and behaviors. For example: a dog is an object having attributes such as color, weight, age, etc. and behaviors such as barking, wagging tail, etc. In OOP, attributes are represented by data and behaviors are represented by functions.
- **Class:** A class is the collection of similar objects. It is defined as the template or prototype to define the common attributes and behavior for all the objects of the class. The entire set of data and code of an object can be made a user-defined data type with the help of a class. In fact objects are variables of type class. Once, a class has been defined, we can create any number of objects associated with that class.
- **Data abstraction:** In OOP, data abstraction defines the conceptual boundaries of an object. These boundaries distinguish an object from another object. So, abstraction is the act of representing essential features without including the background details. It focuses the outside view of an object, separating its essential behavior from its implementation.
- **Encapsulation:** Encapsulation is a way if organizing data and function into a structure by concealing the way the object is implemented, that is preventing access to data by any means other than those specified. Encapsulation therefore guarantees the integrity of the data contained in the object. It implies that there is visibility to the functionalities offered by an object, and no visibility to its data.
- **Inheritance:** The process of creating a new class form an existing class in which objects of the new class inherit the attributes and behaviors of the existing class is known as inheritance. The newly created class is called derived class or child class and the class from which new class is derived is called base class or parent class. It permits the expansion and reuse of existing code without rewriting it hence, the concept of inheritance supports the concept of reusability.
- **Polymorphism:** The meaning of polymorphism is having many forms. The term *poly* means *many* and the term *morph* means to *form.* It is an important feature of OOP which refers to the ability of an object to take on different forms depending upon situations. It simplifies coding and reduces the rework involved in modifying and developing an application.. Then polymorphism concerns the possibility for a single property of exposing multiple possible states. The generally accepted definition for this term in object oriented programming is the capability of objects belonging to the same class hierarchy to react differently to the same method call.

❖ Application of OOP:

- Real Time System Design
- Simulation and Modeling System
- Object Oriented Database
- Client –Server System
- Neural Networking and Parallel Networking
- AI and Expert Systems
- CAD/CAM System

❖ Difference between OOP and Structured Programming Language:

Structured Programming	Object-Oriented Programming
Top-down approach is followed.	Bottom-up approach is followed.
Focus is on algorithm and control flow.	Focus is on object model.
Program is divided into a number of sub-modules, or functions, or procedures.	Program is organized by having a number of classes and objects.
Functions are independent of each other.	Each class is related in a hierarchical manner.
No designated receiver in the function call.	There is a designated receiver for each message passing.
Views data and functions as two separate entities.	Views data and function as a single entity.
Maintenance is costly.	Maintenance is relatively cheaper.
Software reuse is not possible.	Helps in software reuse.
Function call is used.	Message passing is used.
Function abstraction is used.	Data abstraction is used.
Algorithm is given importance.	Data is given importance.
Solution is solution-domain specific.	Solution is problem-domain specific.
No encapsulation. Data and functions are separate.	Encapsulation packages code and data altogether. Data and functionalities are put together in a single entity.
Relationship between programmer and program is emphasized.	Relationship between programmer and user is emphasized.
Data-driven technique is used.	Driven by delegation of responsibilities.

c) Event –Oriented Programming:

Event oriented Programming means that the program everything as a response to an event, instead of a top down program where the ordering of the code determines the order of execution.

Program code based on what happens when the user does something. Events can be mouse moves, mouse clicks or double-clicks, keystrokes, changes made in textboxes, timing based on a timer, etc. Note that this is not an exhaustive list of events, just a small sampling. It is quite a bit different from top-down coding. An event-oriented language implies that an application (the computer program) waits for an event to occur before taking any action. E.g. VB, C++, Java etc.

d) Aspect-Oriented Programming:

In software terms, aspect-oriented programming allows us the ability to apply aspects that alter behavior to classes or objects independent of any inheritance hierarchy. We can then apply these aspects either during runtime or compile time. It is easier to demonstrate AOP by example then to describe it. To start, though, it is important to define four key AOP terms I will use repeatedly:

e)

Joinpoint—Well defined points in code that can be identified.

Pointcut—A way of specifying a Joinpoint by some means of configuration or code.

Advice—A way of expressing a cross cutting action that needs to occur.

Mixin—An instance of a class to be mixed in with the target instance of a class to introduce new behavior.

Aspect-Oriented Programming (AOP) complements OO programming by allowing the developer to dynamically modify the static OO model to create a system that can grow to meet new requirements. Just as objects in the real world can change their states during their lifecycles, an application can adopt new characteristics as it develops. E.g. metrices.

e) Subject-Oriented Programming:

Subject-oriented programming is an enhancement of object-oriented programming that allows decentralized class definition. An application developer who needs new operations associated with classes can implement them him/herself, not by editing existing code for the classes, but as a separate collection of class definitions called a subject. Multiple subjects can be composed to yield a complete suite of applications; Without eliminating the advantages of encapsulation, this approach eliminates the need for class ownership. An application developer can write all the code needed for the application, irrespective of which classes are involved, without

Leaving development requirements on others. The cost of this flexibility is a small run-time overhead on operation calls.

Subject-oriented programming gets its name from the fact that each subject defines a subjective view of objects: the particular operations and internal data that that subject requires the objects to have. Subject-oriented programming supports decentralization in time as well as in space. The developers of an application can program extensions to it as separate subjects to be composed with the base application, perhaps in multiple configurations. Integrated development environment, component of visual programming.

1.2 IDE (Integrated Development Environment) and Visual Programming:

An integrated development environment (IDE) or interactive development environment is a software application that provides comprehensive facilities to computer programmers for software development. An IDE normally consists of a source code editor; build automation tools and a debugger. Several modern IDEs integrate with Intelli-sense coding features.

Some examples of IDEs are: Microsoft Visual Studio, Eclipse, NetBeans, Delphi, JBuilder, Borland C++ Builders, FrontPage, Dreamweaver etc.

❖ Components of Visual Programming:

Visual Basic (VB) is a well-known programming language created and developed by Microsoft. VB is the visual form of BASIC, an earlier language originally developed by Dartmouth professors John Kemeny and Thomas Kurtz.

VP is a graphical representation of the programming elements.

❖ Advantages:

- The structure of the Basic programming language is very simple, particularly as to the executable code.
- VB is not only a language but primarily an integrated, interactive development environment ("IDE").
- The VB-IDE has been highly optimized to support rapid application development ("RAD").
- The graphical user interface of the VB-IDE provides intuitively appealing views for the management of the program structure in the large and the various types of entities (classes, modules, procedures, forms,).
- VB provides a comprehensive interactive and context-sensitive online help system.
- When editing program texts the "IntelliSense" technology informs you in a little popup window about the types of constructs that may be entered at the current cursor location.

❖ Disadvantages:

- Visual basic is a proprietary programming language written by Microsoft, so programs written in Visual basic cannot, easily, be transferred to other operating systems.
- There are some, fairly minor disadvantages compared with C. C has better declaration of arrays – its possible to initialise an array of structures in C at declaration time; this is impossible in VB.
- Larger size of RAM is required.
- Primarily, Visual development environment can be used only with GUI operating system such as windows.
- High speed processor and huge capacity hard disk is required.

❖ Components of VPLs:

Following are the popular components of Visual programming

1. **Window:** A window sometimes also called a form is the most important of all the visual interface components. A window plays the role of the canvas in a painting. Without the canvas there is no painting. Similarly, without the window there is no user interface. The components that make up the user interface have to be placed in the window and cannot exist independent of the window.

2. Buttons: A button is used to initiate an action. The text on the button is indicative of the action that it will initiate. Clicking on a button initiate the action associated with the button.

3. Text boxes: Text boxes are used to accept information from the user. The user interface will display one text for each piece of information

4. List Boxes or Pop-up Lists: List boxes are used to present the user with the possible options. The user can select one or more of the listed options.

5. Label: The label is used to place text in a window. Typically label is used to identify controls that do not have caption properties of their own.

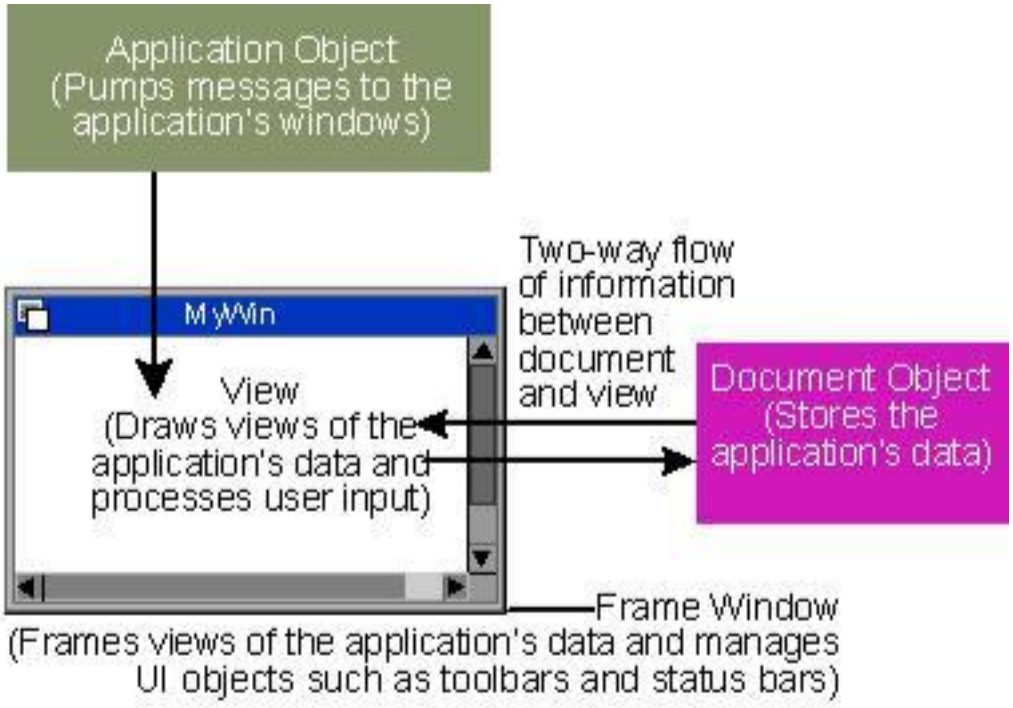
6. Radio button or Option Button: The radio buttons, also referred as option buttons, are used when the user can select only one of multiple options.

1.3 Document-view architecture and its advantages:

The Document/View architecture is the foundation used to create applications based on the Microsoft Foundation Classes library. It allows you to make distinct the different parts that compose a computer program including what the user sees as part of your application and the document a user would work on. This is done through a combination of separate classes that work as an ensemble.

From MFC 4.0 the document view architecture came into action. MFC supports two types of document/view applications.

The first is single-document interface (SDI) applications, which support just one open document at a time. The second is multiple-document interface (MDI) applications, which permit the user to have two or more documents open concurrently. MDI supports multiple views.



The parts that compose the Document/View architecture are a frame, one or more documents, and the view. Put together, these entities make up a usable application.

View:

A view is the platform the user is working on to do his or her job. To let the user do anything on an application, you must provide a view, which is an object based on the CView class. You can either directly use one of the classes derived from CView or you can derive your own custom class from CView or one of its child classes.

Document:

A document is similar to a bucket. For a computer application, a document holds the user's data. To create the document part of this architecture, you must derive an object from the CDocument class.

Frame:

As the name suggests, a frame is a combination of the building blocks, the structure, and the borders of an item. A frame gives "physical" presence to a window. It also defines the location of an object with regards to the Windows desktop.

Key Classes of Document/View Architecture:

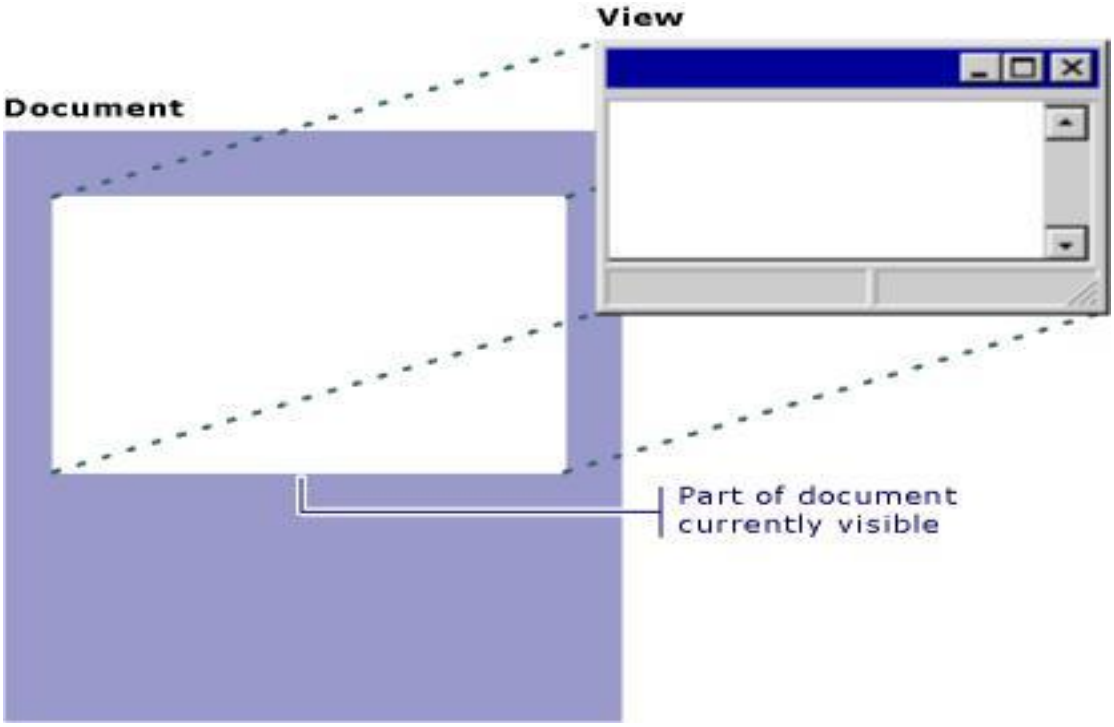
The **CDocument** (or COleDocument) class supports objects used to store or control our program's data and provides the basic functionality for programmer-defined document classes. A document represents the unit of data that the user typically opens with the Open command on the File menu and saves with the Save command on the File menu.

The **CView** (or one of its many derived classes) provides the basic functionality for programmer-defined view classes. A view is attached to a document and acts as an intermediary between the document and the user: the view renders an image of the document on the screen and interprets user input as operations upon the document. The view also renders the image for both printing and print preview.

CFrameView (or one of its variations) supports objects that provides the frame around one or more views of a document.

CDocTemplate (or CSingleDocTemplate or CMultiDocTemplate) supports an object that coordinates one or more existing documents of a given type and manages creating the correct document, view, and frame window objects for that type.

The following figure shows the relationship between a document and its view.



- The document/view implementation in the class library separates the data itself from its display and from user operations on the data.
- All changes to the data are managed through the document class. The view calls this interface to access and update the data.

A document template creates documents, their associated views, and the frame windows that frame the views. The document template is responsible for creating and managing all documents of one document type.

Advantages of the Document/View Architecture:

The key advantage to using the MFC document/view architecture is that the architecture supports multiple views of the same document particularly well. Suppose our application lets users view numerical data either in spreadsheet form or in chart form. A user might want to see simultaneously both the raw data, in spreadsheet form, and a chart that results from the data. We display these separate views in separate frame windows or in splitter panes within a single window. Now suppose the user can edit the data in the spreadsheet and see the changes instantly reflected in the chart.

1.4 Virtual machines and runtime environments:

A virtual machine (VM) is a software program or operating system that not only exhibits the behaviour of a separate computer, but is also capable of performing tasks such as running applications and programs like a separate computer. A virtual machine, usually known as a guest is created within another computing environment referred as a "host." Multiple virtual machines can exist within a single host at one time. There are many types of VM such as System virtual machines and process virtual machines.

❖ Run Time Environment(RTE):

A runtime environment is the execution environment provided to an application or software by the operating system. In a runtime environment, the application can send instructions or commands to the processor and access other system resources such as RAM, which otherwise is not possible as most programming languages used are high level languages

Important question from this Unit:

Q.1) Explain Document view Architecture with its advantages?

Q.2) Explain About significance or features of aspect oriented and subject oriented Programming with example?

Q.3) Differentiate between object oriented and event oriented?

Q.4) Describe the feature of event, object, subject, procedure oriented with example?

Q.5) what is IDE and What are the components of Visual Programming?